

Course-Level AI Use Case Integration Plan

Institute:	K J Somaiya School of Engineering (KJSSE)
Department:	Computer Engineering
Program:	B.Tech. (Computer Engineering)
Scheme:	SVU-2025 As per Scheme NEP 2020, (With Effect from 2026-2027)
Semester:	III
Course Type:	Practical Core (Laboratory Course)
Course Name:	Web Development Laboratory
Credits (L-T-P):	0 - 0 - 1
Course Code:	316U01L306
Document Version:	V1 - 07/07/2026
Implementation Year:	A.Y. 2026-27
Course In-charge(s):	Dr. Pradnya Gotmare, Sheetal Pareira, Rohini Nair, Shivani Deosthale, Veena Badgujar

AI Use Cases Integration (High: Major Integration, Medium: Moderate Integration, Low: Limited Integration, NA: No Integration):

AI Use Cases Integration: (High: Major Integration, Medium: Moderate Integration, Low: Limited Integration, NA: No Integration)	ID	Mapping	SDGs	Use Case PI
	KJS-CES-01	High	13, 3, 11, 9	Dr. Radhika Kotecha
	KJS-AGR-01	High	2, 13, 12, 15	Dr. Shyamal Virnodkar
	KJS-HLC-01	High	3, 9, 10	Dr. Ninad Mehendale
	KJS-SRS-01	N/A	9, 17	Dr. Umesh Shinde
	KJS-CES-02	N/A	7, 13, 9, 11	Dr. Bhusnoor
Forwarding Head:	Dr. Deepak Sharma, HOD Computer Engineering Department			

Teaching-Learning Integration Summary

AI Use Case	Course Contribution to AI Use Case	Relevant Modules / Topics / Experiments from Syllabus	Integration Approach
Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning (KJS-CES-01)	Design and development of the front-end and full-stack web interface (static pages, responsive layouts, client-side scripting, and React-based single page application with routing, authentication, and API integration) for the Heatwave Intelligence dashboard, alert/advisory display, and stakeholder-facing decision-support portal.	Fundamentals of Web Technology, Layout Designing (HTML5 & CSS3), JavaScript & jQuery for Front-end, React Part-I (Components, JSX, Hooks), React Part-II (Routing, Authentication, Mock API, Node/Express Full-Stack).	Use Case-based Illustrations, Front-end Prototyping, Hands-on Laboratory Exercises, Component-based Development, API Integration Exercises, and Full-Stack Mini-Project.
Irrigation Advisory System for Sugarcane Crop using AI and Sensor-based Technology (KJS-AGR-01)	Design and development of the front-end and full-stack web interface (static pages, responsive layouts, client-side scripting, and React-based single page application with routing, authentication, and API integration) for the Irrigation Advisory dashboard, plot-specific soil-moisture and irrigation recommendation display, and farmer/agronomist-facing decision-support portal.	Fundamentals of Web Technology, Layout Designing (HTML5 & CSS3), JavaScript & jQuery for Front-end, React Part-I (Components, JSX, Hooks), React Part-II (Routing, Authentication, Mock API, Node/Express Full-Stack).	Use Case-based Illustrations, Front-end Prototyping, Hands-on Laboratory Exercises, Component-based Development, API Integration Exercises, and Full-Stack Mini-Project.
Urine Test Strip Reader using Colorimetry as a Point-of-Care Technology (KJS-HLC-01)	Design and development of the front-end and full-stack web interface (static pages, responsive layouts, client-side scripting, and React-based single page application with routing, authentication, and API integration) for the Urine Test Strip Reader clinician dashboard, patient analyte-report display, and clinician/pathologist-	Fundamentals of Web Technology, Layout Designing (HTML5 & CSS3), JavaScript & jQuery for Front-end, React Part-I (Components, JSX, Hooks), React Part-II (Routing, Authentication, Mock API, Node/Express Full-Stack).	Use Case-based Illustrations, Front-end Prototyping, Hands-on Laboratory Exercises, Component-based Development, API Integration Exercises, and Full-Stack Mini-Project.

	facing decision-support portal.		

Guidelines: The AI Use Case Integration refers to the use of a real-world AI-based solution as the practical context for teaching course concepts, rather than teaching them in isolation. Depending on the nature of the course, this integration may involve explaining concepts through the Use Case (which is an AI-based solution), demonstrating how the AI-based solution applies those concepts, analyzing the components of the AI solution, implementing relevant parts of the solution in laboratory sessions, and extending or enhancing the solution through assignments or other learning activities.

Detailed Plan of AI Use Case Integration in Teaching-Learning

AI Use Case: Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning (KJS-CES-01)

A. AI Use Case-based Teaching Plan:

Module Name	Topic(s)	AI Use Case Integration Details (How the Topic will be Taught using the AI Use Case)
1. Fundamentals of Web Technology	Internet and Web browser fundamentals, HTTP(S) Protocol, Client-server architecture, request-response cycle	Explain the client-server architecture and HTTP(S) request-response cycle using the Heatwave Intelligence System: AWS sensors and the IMD database act as data sources on the server side, while meteorologists, disaster-management officers, and citizens access forecasts, hotspot maps, and advisories through browser-based dashboard requests.
	Introduction to web browsers and rendering	Discuss how a browser renders the Heatwave dashboard page (HTML/CSS/JS) delivered by the server, and how repeated dashboard refresh cycles display updated temperature and alert data.
	Version control with Git & GitHub: Introduction, basic commands	Use Git & GitHub to manage the source code of the Heatwave dashboard front-end as a team project - creating repositories, committing incremental UI features (region view, alert banner, advisory form), branching, and merging contributions from multiple student developers.
2. Layout Designing (HTML5 and CSS3)	HTML: The development process, Html tags and simple HTML	Build the static HTML skeleton of the Heatwave dashboard homepage, including sections for current temperature, region selector, and alert summary.
	Web site structure, Tables, forms, Media (Image, Audio, Video), Html5 New Elements, Responsive elements	Use HTML tables to display region-wise temperature and forecast data, forms to capture advisory-subscription details (name, region, contact), and media/icons to represent weather conditions and heatwave severity levels.
	CSS3: Introduction to CSS And Its Use In Websites, Properties, Background, Text, Layout, Selectors, Elements, Attributes, Pseudo Classes	Style the dashboard with a responsive, colour-coded layout - e.g., green/yellow/orange/red backgrounds for heatwave severity levels, a navigation bar for switching between regions/seasons, and pseudo-classes for hover effects on hotspot markers.
	Animation, Animation Effects	Apply CSS animation to make critical heatwave alert banners pulse or flash, drawing attention to severe warnings on the early-warning dashboard.

3. JavaScript for Front-end (JavaScript and jQuery)	Javascript: values, variables, operators, expressions, message printing methods, control statements	Write JavaScript logic that classifies a given maximum temperature reading into a heatwave severity category (Normal / Heatwave / Severe Heatwave) using conditional statements, and displays the corresponding message on the dashboard.
	object-oriented JS, functions, Arrays, String, Regular Expression	Model weather observations as JavaScript objects and arrays (station, region, timestamp, max temperature); write functions to filter/sort observations by region or severity, and use regular expressions to validate station-ID / pin-code formats entered by users.
	Form validation, Form events-mouse and key event, cookie management	Validate the heatwave advisory subscription form (name, email, mobile number, region) using form events, and store the citizen's last-selected region as a cookie so the dashboard remembers their preference on return visits.
	jQuery: Introduction, Basics, Selectors, DOM Traversing, Event handling	Use jQuery selectors and event handling to dynamically update the alert-banner and temperature-card sections of the dashboard whenever a new heatwave alert or forecast value is simulated, without reloading the page.
4. REACT Part-I	React Introduction, Install node, Create an app using create-react-app, basics of react app	Scaffold the Heatwave Intelligence dashboard as a React single-page application using create-react-app, structuring it into a component-based front-end.
	JSX, virtual DOMS, Single page apps, React Lifecycle, States and props	Design components such as RegionCard, ForecastChart, and AlertBanner using JSX; pass forecast and hotspot data as props, and use component state and lifecycle methods to manage the currently selected region and season.
	Functional and class components, Hooks	Implement functional components with hooks (useState, useEffect) to periodically refresh AWS observation data and re-render updated temperature and alert values on the dashboard.
	Basic forms, Event Handling	Build controlled form components for filtering the dashboard by region, date range, or severity level, handling change and submit events in React.
5. REACT Part-II	React routing, Single page app	Implement React Router to provide multiple dashboard views - Home (overview), Region Detail (forecast & AWS readings), and Advisory History - within a single-page Heatwave Intelligence application.

	Authentication and Authorization in Full-Stack Applications	Implement role-based authentication and authorization so that Meteorologists, DBAs, Disaster Management Officers, and public Dashboard Users each see and can perform only the actions relevant to their role (e.g., only authorised roles can approve/publish advisories).
	Creating a Mock API Server Axios, Server-side rendering	Create a mock API server (e.g., using Json-server) exposing endpoints for weather observations, forecasts, hotspots, and advisories, and use Axios in React to fetch and display this data on the dashboard.
	Node basics, express, and full stack application	Develop a Node.js/Express backend that serves heatwave forecast, alert, and advisory APIs (backed by the meteorological database) to the React front-end, completing a full-stack Heatwave Intelligence dashboard application.

B. AI Use Case-based Lab Plan:

Experiment Title / Module	AI Use Case-based Practical Activity (How the Experiment will be Performed using the AI Use Case)
1. Design of a Static Webpage using HTML5 and CSS3	Design the static homepage of the Heatwave Intelligence dashboard using HTML5 tags, tables, and forms to display current region-wise temperatures, and CSS3 to style backgrounds, text, and layout for readability of heatwave information.
2. Design of a Responsive Layout using CSS3	Build a responsive layout for the Heatwave dashboard (using CSS3/Bootstrap or Tailwind) so that the region-wise temperature cards, hotspot summary, and alert banners adapt correctly across desktop, tablet, and mobile screens used by field officers and citizens.
3. Implementation of Client-side Form Validation using JavaScript	Implement JavaScript-based validation for the heatwave advisory subscription form (name, email, mobile number, region) and use cookies to remember a citizen's preferred region across visits.
4. Implementation of Dynamic DOM Updates using jQuery	Use jQuery selectors, DOM traversing, and event handling to dynamically refresh temperature values and alert banners on the dashboard when new AWS/forecast data is simulated.
5. Version Control of a Web Project using Git & GitHub	Set up a GitHub repository for the Heatwave dashboard front-end, and practice commits, branching, and merging while incrementally adding dashboard features (region selector, alert banner, advisory form) in a team.
6. Development of React Components for Data Display	Develop React components (RegionCard, ForecastChart, AlertBanner) that receive heatwave forecast, hotspot, and severity data as props and render them on the dashboard.

7. State Management and Hooks in React	Use React state and hooks (useState, useEffect) to manage the selected region/season and to periodically refresh AWS observation data displayed on the dashboard.
8. Implementation of Forms and Event Handling in React	Build a React-based filter form allowing users to filter the Heatwave dashboard by region, season, or severity level, handling form events within React components.
9. Implementation of Client-side Routing using React Router	Implement React Router to create multiple views of the Heatwave Intelligence application - Home/Overview, Region Detail, and Advisory History - within a single-page application.
10. Implementation of Authentication and Role-based Authorization	Implement a login mechanism and role-based access control so that Meteorologist, DBA, Disaster Management Officer, and public Dashboard User roles see appropriate views and permissions (e.g., advisory approval restricted to authorised roles).
11. Creation of a Mock API Server and Data Fetching using Axios	Create a mock API server (e.g., json-server) exposing endpoints for weather observations, forecasts, hotspots, and advisories, and use Axios in the React front-end to fetch and render this data.
12. Development of a Node.js/Express Backend for Full-Stack Integration	Develop Node.js/Express APIs that serve heatwave forecast, alert, and advisory data (representing the meteorological database layer) to the React dashboard, and integrate front-end and back-end into a complete full-stack Heatwave Intelligence Dashboard mini-project.

AI Use Case:
Irrigation Advisory System for Sugarcane Crop using AI and

Sensor-based Technology (KJS-AGR-01)

A. AI Use Case-based Teaching Plan:

Module Name	Topic(s)	AI Use Case Integration Details <i>(How the Topic will be Taught using the AI Use Case)</i>
1. Fundamentals of Web Technology	Internet and Web browser fundamentals, HTTP(S) Protocol, Client-server architecture, request-response cycle	Explain the client-server architecture and HTTP(S) request-response cycle using the Irrigation Advisory System: soil-moisture/weather sensors, IoT gateways, and the agricultural database act as data sources on the server side, while farmers, agronomists, cane-department supervisors, and factory management access plot-specific irrigation advisories through browser-based dashboard requests.
	Introduction to web browsers and rendering	Discuss how a browser renders the Irrigation Advisory dashboard page (HTML/CSS/JS) delivered by the server, and how periodic dashboard refresh cycles display updated soil-moisture readings, weather forecasts, and irrigation recommendations for each geofenced plot.
	Version control with Git & GitHub: Introduction, basic commands	Use Git & GitHub to manage the source code of the Irrigation Advisory dashboard front-end as a team project - creating repositories, committing incremental UI features (plot map, irrigation-schedule card, fertigation panel), branching, and merging contributions from multiple student developers.
2. Layout Designing (HTML5 and CSS3)	HTML: The development process, Html tags and simple HTML	Build the static HTML skeleton of the Irrigation Advisory dashboard homepage, including sections for plot summary, next-irrigation-date, and crop water-requirement status.
	Web site structure, Tables, forms, Media (Image, Audio, Video), Html5 New Elements, Responsive elements	Use HTML tables to display plot-wise soil-moisture, weather, and irrigation-duration data, forms to capture farmer/plot registration details (name, plot ID, crop stage, region), and media/icons to represent geofenced plot maps and water-stress severity levels.
	CSS3: Introduction to CSS And Its Use In Websites, Properties, Background, Text, Layout, Selectors, Elements, Attributes, Pseudo Classes	Style the dashboard with a responsive, colour-coded layout - e.g., green/yellow/orange/red backgrounds for water-stress severity, a navigation bar for switching between plots/farms, and pseudo-classes for hover effects on geofenced plot markers on the map.

	Animation, Animation Effects	Apply CSS animation to make critical 'Irrigate Now' or delayed-irrigation alert banners pulse or flash, drawing attention to time-sensitive advisories on the farmer dashboard.
3. JavaScript for Front-end (JavaScript and jQuery)	Javascript: values, variables, operators, expressions, message printing methods, control statements	Write JavaScript logic that classifies a given soil-moisture/water-stress reading into an irrigation status category (No Irrigation Needed / Irrigate Soon / Irrigate Now) using conditional statements, and displays the corresponding message on the dashboard.
	object-oriented JS, functions, Arrays, String, Regular Expression	Model plot and sensor observations as JavaScript objects and arrays (plot ID, farmer, crop stage, soil moisture, timestamp); write functions to filter/sort plots by region or irrigation urgency, and use regular expressions to validate plot-ID / pin-code formats entered by users.
	Form validation, Form events-mouse and key event, cookie management	Validate the farmer/plot registration and advisory-subscription form (name, mobile number, plot ID, region) using form events, and store the farmer's last-selected plot as a cookie so the dashboard remembers their preference on return visits.
	jQuery: Introduction, Basics, Selectors, DOM Traversing, Event handling	Use jQuery selectors and event handling to dynamically update the irrigation-schedule and advisory-banner sections of the dashboard whenever a new soil-moisture or weather reading is simulated, without reloading the page.
4. REACT Part-I	React Introduction, Install node, Create an app using create-react-app, basics of react app	Scaffold the Irrigation Advisory dashboard as a React single-page application using create-react-app, structuring it into a component-based front-end.
	JSX, virtual DOMS, Single page apps, React Lifecycle, States and props	Design components such as PlotCard, SoilMoistureChart, and AdvisoryBanner using JSX; pass plot, sensor, and advisory data as props, and use component state and lifecycle methods to manage the currently selected plot and crop stage.
	Functional and class components, Hooks	Implement functional components with hooks (useState, useEffect) to periodically refresh soil-moisture and weather-sensor data and re-render updated irrigation recommendations on the dashboard.
	Basic forms, Event Handling	Build controlled form components for filtering the dashboard by farm, crop stage, or irrigation urgency, handling change and submit events in React.

5. REACT Part-II	React routing, Single page app	Implement React Router to provide multiple dashboard views - Home (overview), Plot Detail (soil-moisture, weather & recommendation), and Advisory History - within a single-page Irrigation Advisory application.
	Authentication and Authorization in Full-Stack Applications	Implement role-based authentication and authorization so that Farmers, Agronomists, Cane-Department Supervisors, and Factory Management each see and can perform only the actions relevant to their role (e.g., only agronomists can approve/publish irrigation and fertigation advisories).
	Creating a Mock API Server Axios, Server-side rendering	Create a mock API server (e.g., using json-server) exposing endpoints for soil-moisture readings, weather forecasts, plot details, and irrigation/fertigation recommendations, and use Axios in React to fetch and display this data on the dashboard.
	Node basics, express, and full stack application	Develop a Node.js/Express backend that serves irrigation, fertigation, and yield-prediction APIs (backed by the agricultural database) to the React front-end, completing a full-stack Irrigation Advisory Dashboard application.

B. AI Use Case-based Lab Plan:

Experiment Title / Module	AI Use Case-based Practical Activity (How the Experiment will be Performed using the AI Use Case)
1. Design of a Static Webpage using HTML5 and CSS3	Design the static homepage of the Irrigation Advisory dashboard using HTML5 tags, tables, and forms to display plot-wise soil-moisture and weather data, and CSS3 to style backgrounds, text, and layout for readability of irrigation information.
2. Design of a Responsive Layout using CSS3	Build a responsive layout for the Irrigation Advisory dashboard (using CSS3/Bootstrap or Tailwind) so that the plot summary cards, soil-moisture status, and advisory banners adapt correctly across desktop, tablet, and mobile screens used by field officers and farmers.
3. Implementation of Client-side Form Validation using JavaScript	Implement JavaScript-based validation for the farmer/plot registration and advisory-subscription form (name, mobile number, plot ID, region) and use cookies to remember a farmer's preferred/last-viewed plot across visits.
4. Implementation of Dynamic DOM Updates using jQuery	Use jQuery selectors, DOM traversing, and event handling to dynamically refresh soil-moisture values and advisory banners on the dashboard when new sensor/weather data is simulated.
5. Version Control of a Web Project using Git & GitHub	Set up a GitHub repository for the Irrigation Advisory dashboard front-end, and practice commits, branching, and merging while incrementally adding dashboard features (plot map, irrigation-schedule card, fertigation panel) in a team.

6. Development of React Components for Data Display	Develop React components (PlotCard, SoilMoistureChart, AdvisoryBanner) that receive soil-moisture, weather, and irrigation-recommendation data as props and render them on the dashboard.
7. State Management and Hooks in React	Use React state and hooks (useState, useEffect) to manage the selected plot/farm and to periodically refresh soil-moisture and weather-sensor data displayed on the dashboard.
8. Implementation of Forms and Event Handling in React	Build a React-based filter form allowing users to filter the Irrigation Advisory dashboard by farm, crop stage, or irrigation urgency, handling form events within React components.
9. Implementation of Client-side Routing using React Router	Implement React Router to create multiple views of the Irrigation Advisory application - Home/Overview, Plot Detail, and Advisory History - within a single-page application.
10. Implementation of Authentication and Role-based Authorization	Implement a login mechanism and role-based access control so that Farmer, Agronomist, Cane-Department Supervisor, and Factory Management roles see appropriate views and permissions (e.g., advisory approval restricted to authorised roles).
11. Creation of a Mock API Server and Data Fetching using Axios	Create a mock API server (e.g., json-server) exposing endpoints for soil-moisture readings, weather forecasts, plot details, and irrigation/fertigation recommendations, and use Axios in the React front-end to fetch and render this data.
12. Development of a Node.js/Express Backend for Full-Stack Integration	Develop Node.js/Express APIs that serve irrigation, fertigation, and yield-prediction data (representing the agricultural database layer) to the React dashboard, and integrate front-end and back-end into a complete full-stack Irrigation Advisory Dashboard mini-project.

AI Use Case:

Urine Test Strip Reader using Colorimetry as a Point-of-Care Technology (KJS-HLC-01)

A. AI Use Case-based Teaching Plan:

Module Name	Topic(s)	AI Use Case Integration Details (How the Topic will be Taught using the AI Use Case)
1. Fundamentals of Web Technology	Internet and Web browser fundamentals, HTTP(S) Protocol, Client-server architecture, request-response cycle	Explain the client-server architecture and HTTP(S) request-response cycle using the Urine Test Strip Reader system: the Raspberry Pi-based device and hospital database act as data sources on the server side, while clinicians, pathologists, and lab technicians access patient test reports through browser-based dashboard requests.
	Introduction to web browsers and rendering	Discuss how a browser renders the clinician dashboard page (HTML/CSS/JS) delivered by the server, and how dashboard refresh cycles display newly generated analyte-concentration reports and timestamps for each patient.
	Version control with Git & GitHub: Introduction, basic commands	Use Git & GitHub to manage the source code of the clinician dashboard front-end as a team project - creating repositories, committing incremental UI features (patient list, analyte-report view, alert-flag panel), branching, and merging contributions from multiple student developers.
2. Layout Designing (HTML5 and CSS3)	HTML: The development process, Html tags and simple HTML	Build the static HTML skeleton of the clinician dashboard homepage, including sections for the patient list, latest test report, and analyte-summary panel.
	Web site structure, Tables, forms, Media (Image, Audio, Video), Html5 New Elements, Responsive elements	Use HTML tables to display the 10 analyte concentrations (glucose, protein, pH, ketones, blood, etc.) per patient, forms to capture patient/test registration details (patient ID, name, sample timestamp), and media/icons to represent device status and report flags.
	CSS3: Introduction to CSS And Its Use In Websites, Properties, Background, Text, Layout, Selectors, Elements, Attributes, Pseudo Classes	Style the dashboard with a responsive, colour-coded layout - e.g., green/red backgrounds to flag normal vs abnormal analyte values, a navigation bar for switching between patients/tests, and pseudo-classes for hover effects on report rows.
	Animation, Animation Effects	Apply CSS animation to make critical out-of-range analyte alerts (e.g., abnormal glucose or protein) pulse or flash, drawing attention to results requiring immediate clinical review.

3. JavaScript for Front-end (JavaScript and jQuery)	Javascript: values, variables, operators, expressions, message printing methods, control statements	Write JavaScript logic that classifies a given analyte concentration into a result category (Normal / Abnormal / Critical) against clinical reference intervals using conditional statements, and displays the corresponding flag on the dashboard.
	object-oriented JS, functions, Arrays, String, Regular Expression	Model patient test reports as JavaScript objects and arrays (patient ID, analyte values, timestamp); write functions to filter/sort reports by patient or flagged status, and use regular expressions to validate patient-ID formats entered by users.
	Form validation, Form events-mouse and key event, cookie management	Validate the patient/test registration form (patient ID, name, contact, sample time) using form events, and store the clinician's last-viewed patient as a cookie so the dashboard remembers their preference on return visits.
	jQuery: Introduction, Basics, Selectors, DOM Traversing, Event handling	Use jQuery selectors and event handling to dynamically update the analyte-report table and alert-banner sections of the dashboard whenever a new test result is simulated, without reloading the page.
4. REACT Part-I	React Introduction, Install node, Create an app using create-react-app, basics of react app	Scaffold the Urine Test Strip Reader clinician dashboard as a React single-page application using create-react-app, structuring it into a component-based front-end.
	JSX, virtual DOMS, Single page apps, React Lifecycle, States and props	Design components such as PatientCard, AnalyteReportTable, and AlertBanner using JSX; pass patient and analyte data as props, and use component state and lifecycle methods to manage the currently selected patient and test.
	Functional and class components, Hooks	Implement functional components with hooks (useState, useEffect) to periodically refresh incoming test results and re-render updated analyte values and alert flags on the dashboard.
	Basic forms, Event Handling	Build controlled form components for searching/filtering the dashboard by patient ID, date, or flagged status, handling change and submit events in React.
5. REACT Part-II	React routing, Single page app	Implement React Router to provide multiple dashboard views - Home (patient list), Patient Detail (full 10-analyte report), and Test History - within a single-page Urine Test Strip Reader application.

	Authentication and Authorization in Full-Stack Applications	Implement role-based authentication and authorization so that Clinicians/Pathologists, Lab Technicians, and Hospital Administrators each see and can perform only the actions relevant to their role (e.g., only clinicians can confirm/approve a report before it is released).
	Creating a Mock API Server Axios, Server-side rendering	Create a mock API server (e.g., using json-server) exposing endpoints for patient records, analyte concentrations, and test reports, and use Axios in React to fetch and display this data on the dashboard.
	Node basics, express, and full stack application	Develop a Node.js/Express backend that serves patient-record and analyte-report APIs (backed by the hospital-compatible database) to the React front-end, completing a full-stack Urine Test Strip Reader clinician dashboard application.

B. AI Use Case-based Lab Plan:

Experiment Title / Module	AI Use Case-based Practical Activity <i>(How the Experiment will be Performed using the AI Use Case)</i>
1. Design of a Static Webpage using HTML5 and CSS3	Design the static homepage of the Urine Test Strip Reader clinician dashboard using HTML5 tags, tables, and forms to display patient-wise analyte data, and CSS3 to style backgrounds, text, and layout for readability of test reports.
2. Design of a Responsive Layout using CSS3	Build a responsive layout for the clinician dashboard (using CSS3/Bootstrap or Tailwind) so that the patient list, analyte-report table, and alert banners adapt correctly across desktop, tablet, and mobile screens used by clinicians and lab staff.
3. Implementation of Client-side Form Validation using JavaScript	Implement JavaScript-based validation for the patient/test registration form (patient ID, name, contact, sample time) and use cookies to remember a clinician's last-viewed patient across visits.
4. Implementation of Dynamic DOM Updates using jQuery	Use jQuery selectors, DOM traversing, and event handling to dynamically refresh analyte values and alert banners on the dashboard when new test results are simulated.
5. Version Control of a Web Project using Git & GitHub	Set up a GitHub repository for the clinician dashboard front-end, and practice commits, branching, and merging while incrementally adding dashboard features (patient list, analyte-report view, alert-flag panel) in a team.
6. Development of React Components for Data Display	Develop React components (PatientCard, AnalyteReportTable, AlertBanner) that receive patient and analyte data as props and render them on the dashboard.
7. State Management and Hooks in React	Use React state and hooks (useState, useEffect) to manage the selected patient/test and to periodically refresh incoming analyte results displayed on the dashboard.

8. Implementation of Forms and Event Handling in React	Build a React-based filter form allowing users to search/filter the clinician dashboard by patient ID, date, or flagged status, handling form events within React components.
9. Implementation of Client-side Routing using React Router	Implement React Router to create multiple views of the Urine Test Strip Reader application - Home/Patient List, Patient Detail, and Test History - within a single-page application.
10. Implementation of Authentication and Role-based Authorization	Implement a login mechanism and role-based access control so that Clinician/Pathologist, Lab Technician, and Hospital Administrator roles see appropriate views and permissions (e.g., report confirmation restricted to authorised clinicians).
11. Creation of a Mock API Server and Data Fetching using Axios	Create a mock API server (e.g., json-server) exposing endpoints for patient records, analyte concentrations, and test reports, and use Axios in the React front-end to fetch and render this data.
12. Development of a Node.js/Express Backend for Full-Stack Integration	Develop Node.js/Express APIs that serve patient-record and analyte-report data (representing the hospital database layer) to the React dashboard, and integrate front-end and back-end into a complete full-stack Urine Test Strip Reader clinician dashboard mini-project.

AI Use Case:

AI-Based Autonomous Inter-Satellite Data Routing and Multi-Mode Amateur Radio Communication for PocketQube Missions

(KJS-SRS-01)

A. AI Use Case-based Teaching Plan:

Module Name	Topic(s)	AI Use Case Integration Details (How the Topic will be Taught using the AI Use Case)
	<AI Use Case-based Teaching Plan is to be prepared for all Courses. Teaching-Learning shall be executed as discussed in meetings.>	<Include all Modules and Topics from Syllabus in this table, and for any topic that cannot be integrated, specify N/A>
	<Some topics within a module may be grouped, if required>	

B. AI Use Case-based Lab Plan:

Experiment Title / Module	AI Use Case-based Practical Activity (How the Experiment will be Performed using the AI Use Case)
	<Include all Experiments / Modules from Syllabus in this table, and for any Experiment / Module that cannot be integrated, specify N/A>

AI Use Case: AI-Enabled Micro-channel Battery Thermal Management System for Electric Vehicles (KJS-CES-02)

A. AI Use Case-based Teaching Plan:

Module Name	Topic(s)	AI Use Case Integration Details (How the Topic will be Taught using the AI Use Case)
	<AI Use Case-based Teaching Plan is to be prepared for all Courses. Teaching-Learning shall be executed as discussed in meetings.>	<Include all Modules and Topics from Syllabus in this table, and for any topic that cannot be integrated, specify N/A>
	<Some topics within a module may be grouped, if required>	

B. AI Use Case-based Lab Plan:

Experiment Title / Module	AI Use Case-based Practical Activity (How the Experiment will be Performed using the AI Use Case)
	<Include all Experiments / Modules from Syllabus in this table, and for any Experiment / Module that cannot be integrated, specify N/A>